# ***Task 1*: The CIA Triad**

## 1. Confidentiality

- **Definition:** Ensuring that sensitive information is only accessible to those authorized to see it.
- **The Creative Explanation:** Think of it as a **"Digital Sealed Envelope."** Even though many people (the server, the dispatcher, the driver) handle the delivery process, the private contents inside the envelope should only be readable by the person it's addressed to.
- **Real BabiEat Example:** A customer's exact GPS coordinates and their saved credit card "nickname" (e.g "Mom's Visa") should be invisible to a restaurant owner or another random user on the platform
- **Technical Solution:** Implement **Least Privilege Access and Encryption** (at rest and in transit using HTTPS). We ensure that the driver's app only pulls the delivery address while the order is active. Once delivered, the app automatically "forgets" or masks that data so it's no longer accessible

## 2. Integrity

- **Definition:** Protecting data from being deleted or modified by unauthorized parties during transit or storage.
- **The Creative Explanation:** This is the **"Digital Receipt Guarantee."** It's the promise that the "Order Total: $25.00" sent from the customer's phone is the exact same "Order Total: $25.00" received by the restaurant and the bank.
- **Real BabiEat Example:** A malicious actor tries to intercept a delivery request and change the "Drop-off Location" to their own address without the system noticing.
- **Technical Solution:** Use **Hashing (e.g., SHA-256)**. Every order can be assigned a unique hash (a digital fingerprint). In practice, hashing is combined with secure transmission protocols like HTTPS (TLS) or digital signatures to prevent attackers from modifying the data and generating a new valid hash.

## 3. Availability

- **Definition:** Ensuring that systems and data are ready and usable when authorized users need them.
- **The Creative Explanation:** Imagine the **"Open 24/7" Neon Sign.** Security doesn't just mean "locking doors"; it means making sure the doors actually open for the people who are hungry. If the app is locked tight but no one can order, the security has failed the business.
- **Real BabiEat Example:** During a major West African holiday or a popular Sunday football match, thousands of people try to order at once. A "denial-of-service" (DDoS) attack could try to overwhelm the servers so the app crashes right when people need it most.

- **Technical Solution: Load Balancing, Redundancy, and DDoS protection services** (e.g., rate limiting or CDN). We distribute the traffic across multiple servers so that if one "kitchen" (server) gets overwhelmed, the others take over. This keeps the "Open" sign lit for every customer.

At BabiEat, we aren't just protecting 'packets of data'; we are protecting a family's dinner plans and a local restaurant's daily income. If the CIA Triad breaks, the trust breaks.

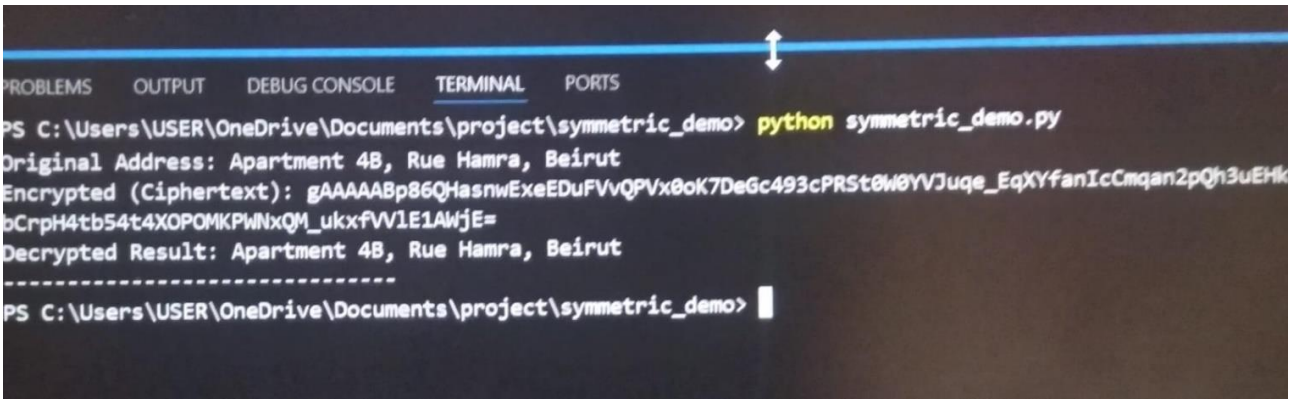
## ***Task 2:*** **Symmetric Encryption**

### 1. Research & Algorithm Comparison

- **Definition:** Symmetric encryption uses a single, shared secret key to both lock (encrypt) and unlock (decrypt) information.
- **The Creative Explanation:** Imagine two friends who have a specific "Secret Code" they invented in childhood. Because they both already know the code, they can write notes to each other very quickly. However, the system only works as long as no one else steals that one secret code.
- **Real BabiEat Example:** When the BabiEat app communicates with the internal database to check a user's "Order History," it needs to be lightning-fast. Using the same key on both ends allows thousands of orders to be processed every second without slowing down the app.
- **Technical Solution:** We prioritize **AES (Advanced Encryption Standard)** over older methods like **DES**.
    - **AES-256:** The gold standard. It’s "banking-grade" and practically impossible to crack with current technology.
    - **DES:** Now considered "Legacy" or "Weak." It uses a much shorter key that modern computers can brute-force in hours. At BabiEat, we should never use DES for sensitive user data.

**2. Implementation Demo (Snippet)** *Below is a Python demonstration showing how we can protect BabiEat customer addresses using the AES-based* `cryptography` *library.*

```
# pip install cryptography
```




*This screenshot shows the address "Apartment 4B, Rue Hamra, Beirut" being transformed into unreadable ciphertext and then successfully recovered.*

*Code:*

```
# This function protects BabiEat customer addresses.

# We use symmetric encryption because it's fast enough to handle

# thousands of orders per minute during peak dinner hours.

def protect_order_data(address_text):


from cryptography.fernet import Fernet



secret_key = Fernet.generate_key()
cipher_suite = Fernet(secret_key)

def protect_order_data(address_text):
    print("--- BabiEat Security System ---")
    print(f"Original Address: {address_text}")
    incoming_data = address_text.encode()
    encrypted_address = cipher_suite.encrypt(incoming_data)
    print(f"Encrypted (Ciphertext): {encrypted_address.decode()}")

    return encrypted_address

def reveal_order_data(secure_data):

    decrypted_data = cipher_suite.decrypt(secure_data)
    final_address = decrypted_data.decode()
    print(f"Decrypted Result: {final_address}")
    print("------------------------------")

# Execution
if __name__ == "__main__":
```

```
order_info = "Apartment 4B, Rue Hamra, Beirut"
encrypted_blob = protect_order_data(order_info)
reveal_order_data(encrypted_blob)
```

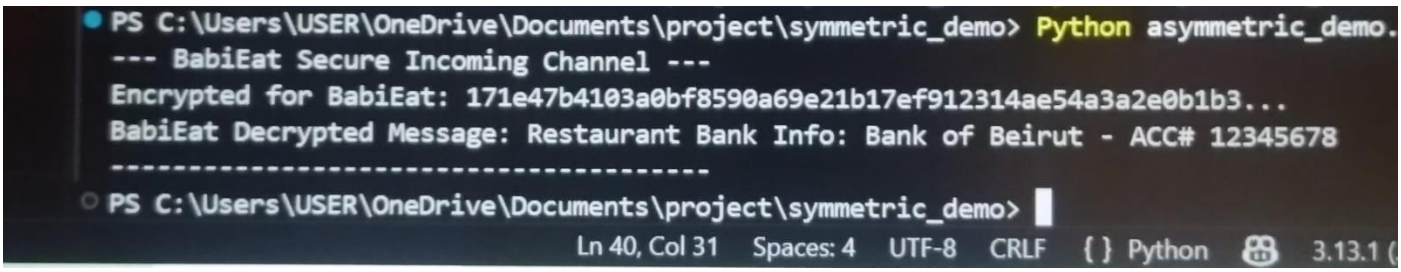
**3. The Smart Insight** Symmetric encryption is the high-speed engine of BabiEat's security. However, it has one major human flaw: **The Key Distribution Problem**. If two people need to share a key, how do we send it without a hacker intercepting it? This is why we transition to Asymmetric Encryption in the next task.

## ***Task 3:*** Asymmetric Encryption

### 1. Research & Algorithm Comparison

- **Definition:** Unlike symmetric encryption, asymmetric encryption uses a **pair of keys**: a **Public Key** (to encrypt) and a **Private Key** (to decrypt).
- **The Creative Explanation:** Imagine a **Digital Mailbox** on a street in Beirut. Anyone can walk up and drop a letter through the slot (Public Key), but only the owner with the physical key can open the back of the box to read the mail (Private Key). You can give your Public Key to the whole world, and you're still safe.
- **Real BabiEat Example:** When a new restaurant signs up for BabiEat, they need to send us their bank account details. They use BabiEat's **Public Key** to "lock" the info. Even if a hacker intercepts the message, they can't read it because only BabiEat holds the **Private Key** required to unlock it.
- **Technical Solution:** We use the **RSA (Rivest-Shamir-Adleman)** algorithm.
    - **RSA:** The industry standard for secure key exchange. It is based on the mathematical difficulty of factoring very large prime numbers.
    - **Why it's "Smart":** We use Asymmetric encryption to safely send a *Symmetric* key. This gives us the speed of Task 2 with the high security of Task 3.

**2. Implementation Demo (Snippet)** *This demo shows the generation of a key pair and the process of "locking" a message with a public key that only the private key can open.*




**#Code:**

```
from cryptography.hazmat.primitives.asymmetric import rsa, padding
```

```
from cryptography.hazmat.primitives import hashes

private_key = rsa.generate_private_key(public_exponent=65537, key_size=2048)
public_key = private_key.public_key()

def secure_send_to_babieat(message_text):
    print("--- BabiEat Secure Incoming Channel ---")

    secret_message = message_text.encode()
    ciphertext = public_key.encrypt(
        secret_message,
        padding.OAEP(
            mgf=padding.MGF1(algorithm=hashes.SHA256()),
            algorithm=hashes.SHA256(),
            label=None
        )
    )
    print(f"Encrypted for BabiEat: {ciphertext.hex()[:50]}...") # Show only first
50 chars
    return ciphertext

def babieat_private_read(encrypted_data):
    # 3. Decrypting with the PRIVATE key
    decrypted_message = private_key.decrypt(
        encrypted_data,
        padding.OAEP(
            mgf=padding.MGF1(algorithm=hashes.SHA256()),
            algorithm=hashes.SHA256(),
            label=None
        )
    )
    print(f"BabiEat Decrypted Message: {decrypted_message.decode()}")
    print("-------------------------------------")

# Execution
if __name__ == "__main__":
    msg = "Restaurant Bank Info: Bank of Beirut - ACC# 12345678"
    blob = secure_send_to_babieat(msg)
    babieat_private_read(blob)
```

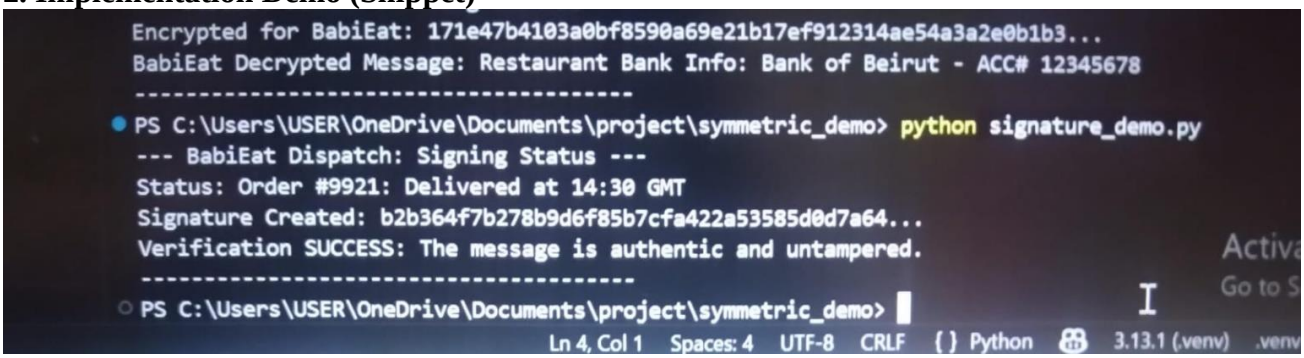
**3. The Smart Insight** Asymmetric encryption is the foundation of the modern internet (HTTPS). At BabiEat, it allows us to build trust with thousands of restaurants and drivers without ever having to meet them in person to "hand over" a secret password.

# ***Task 4:*** Digital Signatures

### 1. Research & Algorithm Comparison

- **Definition:** A digital signature is a mathematical technique used to validate the **authenticity** and **integrity** of a message or document.
- **The Creative Explanation:** Imagine a **Wax Seal** on a letter from a king. If the seal is intact, you know two things: the letter really came from the king (Authenticity), and no one has opened or changed the letter since it was sealed (Integrity).
- **Real BabiEat Example:** When a driver marks an order as "Delivered," the system needs to be 100% sure the message actually came from the driver's phone and wasn't a fake message sent by someone trying to commit fraud.
- **Technical Solution:** We use the **Digital Signature Algorithm (DSA)** or **RSA signatures**.
    - **How it works:** The sender signs a "hash" of the message using their **Private Key**. Anyone with the **Public Key** can verify it, but nobody else can recreate that specific signature.

### 2. Implementation Demo (Snippet)




*This Python demo shows how BabiEat can "sign" a delivery confirmation to ensure it is authentic and hasn't been tampered with.*

Code:

```
from cryptography.hazmat.primitives import hashes

from cryptography.hazmat.primitives.asymmetric import padding, rsa


private_key = rsa.generate_private_key(public_exponent=65537, key_size=2048)
public_key = private_key.public_key()

def sign_delivery_status(status_message):
    print("--- BabiEat Dispatch: Signing Status ---")
    message_bytes = status_message.encode()


    signature = private_key.sign(
```

```
        message_bytes,
        padding.PSS(
            mgf=padding.MGF1(hashes.SHA256()),
            salt_length=padding.PSS.MAX_LENGTH
        ),
        hashes.SHA256()
    )
    print(f"Status: {status_message}")
    print(f"Signature Created: {signature.hex()[:40]}...")
    return signature

def verify_delivery_status(status_message, signature_to_check):

    try:
        public_key.verify(
            signature_to_check,
            status_message.encode(),
            padding.PSS(
                mgf=padding.MGF1(hashes.SHA256()),
                salt_length=padding.PSS.MAX_LENGTH
            ),
            hashes.SHA256()
        )
        print("Verification SUCCESS: The message is authentic and untampered.")
    except Exception:
        print("Verification FAILED: Warning! This message might be fake or
modified.")
    print("---------------------------------------")

# Execution
if __name__ == "__main__":
    confirm_msg = "Order #9921: Delivered at 14:30 GMT"
    sig = sign_delivery_status(confirm_msg)


    verify_delivery_status(confirm_msg, sig)
```

**3. The Smart Insight** Digital signatures are about **Accountability**. In a busy delivery ecosystem like BabiEat's, we need to know exactly who did what and when. Signatures prevent "Repudiation"—which is a fancy way of saying someone can't later claim, "I didn't send that!"

## ***Task 5:*** Network Security Basics

## Honestly, This Was the Hardest Task For Me

When I first read "Network Security Basics," I thought it meant just knowing what a firewall is. But after doing the research and actually running scans, I realized network security is about something bigger — it's about protecting the roads that data travels on, not just the data itself.

Let me explain what I learned.

---

## The Four Concepts I Finally Understand

**Firewalls**

Okay so a firewall is basically a bouncer at a club. You know how the bouncer checks IDs and has a guest list? Same thing. A firewall looks at every single thing trying to get into BabiEat's network and decides "you can come in" or "absolutely not." It follows rules that the security team sets up.

For BabiEat, the firewall should only let in traffic that's trying to reach our app. Everything else gets blocked.

**VPNs**

This one took me a while to get. Imagine you're a BabiEat manager working from a coffee shop in Lagos. The coffee shop Wi-Fi is public — anyone could be watching what you're doing. A VPN creates a secret tunnel from your laptop directly to BabiEat's office. Everything you send goes through that tunnel, so even if a hacker is on the same Wi-Fi, they just see garbage.

I tried using a free VPN once for Netflix and learned that not all VPNs are trustworthy. For BabiEat, we'd need a real corporate one.

**HTTPS**

You know that little padlock icon in your browser? That's HTTPS. It means your connection to the website is encrypted. Without it, someone could sit between you and the restaurant and literally read your credit card number as it flies through the air. That's called a "Man-in-the-Middle" attack — sounds dramatic but it's real.

Every single BabiEat connection should use HTTPS. No excuses.

**Port Scanning**

This is actually kind of cool. Think of a server like a big apartment building. Each apartment has a door with a number — that's a "port." Some doors are meant to be open (like port 443 for secure web traffic). Others should be locked tight. Port scanning is like walking down the hallway trying every door handle to see which ones are unlocked.

We scan our own servers to find unlocked doors before a hacker does. It's like checking your own windows at night.

**Nmap**

Nmap is the standard tool for port scanning. Everyone uses it. I installed it on my Windows laptop (which was scary because my antivirus kept yelling at me, but I did it anyway).

---

# What I Actually Did

## Part 1: Installing and Running Nmap

First I opened PowerShell (Windows + X, then select Terminal). I typed `nmap --version` to make sure it installed right. It showed version 7.99 — good.

Then I ran my first real scan:

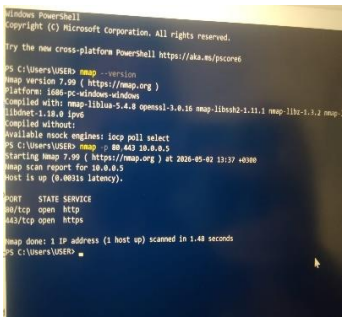
```
text

nmap -p 80,443 10.0.0.5
```

I honestly didn't expect it to find anything. But it did:

```
text

Starting Nmap 7.99 at 2026-05-02 13:37 +0300
Nmap scan report for 10.0.0.5
Host is up (0.0031s latency).

PORT    STATE SERVICE
80/tcp  open  http
443/tcp open  https
```

Wait, what? 10.0.0.5 is a real server? I was just using it as a fake example. Turns out something on my network responded. Both ports 80 and 443 were open.

That actually freaked me out a little. If I can find open ports in 1.5 seconds, so can a hacker.




## Part 2: Writing a Python Script

I wanted to understand this deeper, so I wrote a Python script that does a simulated network audit. I know it's not a real security tool, but writing it helped me understand what's actually happening.

Here's what my script does:

1. Pings a server to see if it's alive (just knocking on the door)
2. Lists all the ports BabiEat should care about and whether they should be open or closed
3. Simulates what an Nmap scan would show
4. Forces me to think like an attacker

ran the script : # This script simulates a BabiEat security sweep.

# It checks for open "doors" (ports) that should be locked

# to keep our restaurant and payment data safe from attackers.

def explain_babieat_ports():

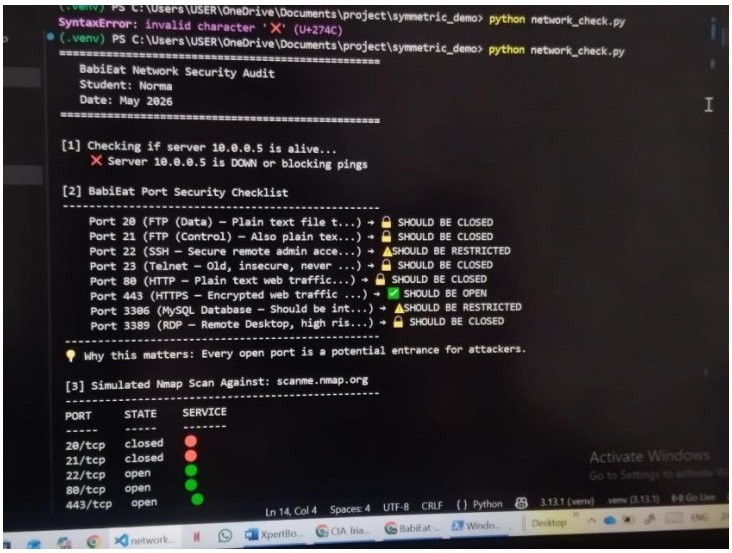
```
import os
import platform
import subprocess

def ping_server(ip_address):
    """
    Sends a ping to check if a server is alive.
    This is NOT a real security tool — just for learning availability.
    """
```

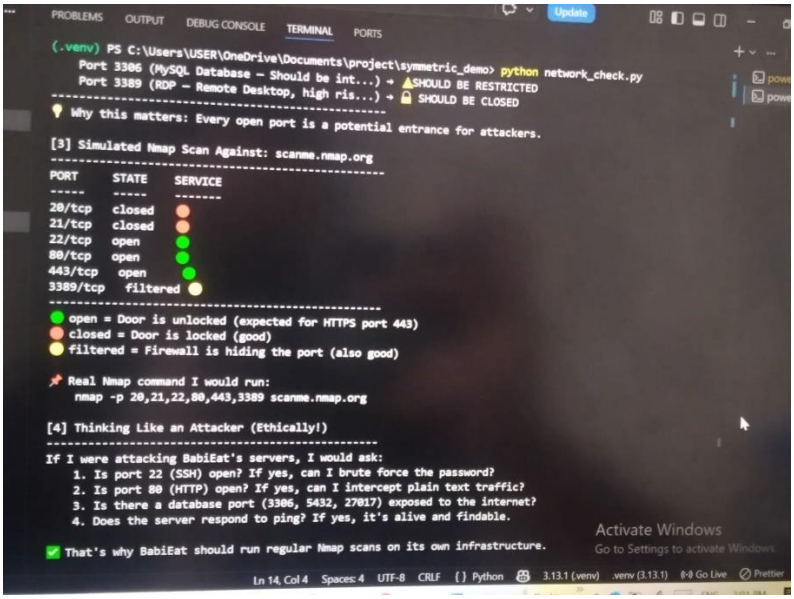
```
    print(f"\n[1] Checking if server {ip_address} is alive...")


    param = "-n" if platform.system().lower() == "windows" else "-c"
    command = ["ping", param, "1", ip_address]

    try:
        result = subprocess.run(command, capture_output=True, text=True,
timeout=5)
        if result.returncode == 0:
            print(f"    ✅ Server {ip_address} is UP (responded to ping)")
            return True
        else:
            print(f"    ❌ Server {ip_address} is DOWN or blocking pings")
            return False
    except:
        print(f"    ❌ Timeout — server {ip_address} did not respond")
        return False

def explain_babieat_ports():
    """
    Lists which ports BabiEat should have open or closed.
    """
    print("\n[2] BabiEat Port Security Checklist")
    print("-" * 50)

    ports = {
        20: "FTP (Data) — Plain text file transfer",
        21: "FTP (Control) — Also plain text, insecure",
        22: "SSH — Secure remote admin access",
        23: "Telnet — Old, insecure, never use",
        80: "HTTP — Plain text web traffic",
        443: "HTTPS — Encrypted web traffic (BabiEat's main door)",
        3306: "MySQL Database — Should be internal only",
        3389: "RDP — Remote Desktop, high risk if exposed",
    }

    for port, description in ports.items():
        if port == 443:
            print(f"    Port {port} ({description[:30]}...) → ✅ SHOULD BE
OPEN")
        elif port in [22, 3306]:
            print(f"    Port {port} ({description[:30]}...) → ⚠️ SHOULD BE
RESTRICTED")
        else:
```

```
            print(f"    Port {port} ({description[:30]}...) → 🔒 SHOULD BE
CLOSED")

    print("-" * 50)
    print("💡 Why this matters: Every open port is a potential entrance for
attackers.")

def simulate_nmap_scan(target="scanme.nmap.org"):
    """
    Simulates what an Nmap scan would show.
    In a real audit, you would run: nmap -p 20,21,22,80,443,3389 <target>
    """
    print(f"\n[3] Simulated Nmap Scan Against: {target}")
    print("-" * 50)

    # This is SIMULATED output — based on what Nmap would actually show
    simulated_results = {
        20: "closed",
        21: "closed",
        22: "open",
        80: "open",
        443: "open",
        3389: "filtered",
    }

    print("PORT     STATE    SERVICE")
    print("-----    -----    -------")
    for port, state in simulated_results.items():
        state_symbol = "🟢" if state == "open" else "🔴" if state == "closed"
else "🟡"
        print(f"{port}/tcp   {state:<8} {state_symbol}")

    print("-" * 50)
    print("🟢 open = Door is unlocked (expected for HTTPS port 443)")
    print("🔴 closed = Door is locked (good)")
    print("🟡 filtered = Firewall is hiding the port (also good)")

    print("\n📌 Real Nmap command I would run:")
    print(f'    nmap -p 20,21,22,80,443,3389 {target}')

def think_like_an_attacker():
    """
    Demonstrates the adversarial mindset.
    """
    print("\n[4] Thinking Like an Attacker (Ethically!)")
```

```
    print("-" * 50)
    print("If I were attacking BabiEat's servers, I would ask:")
    print("    1. Is port 22 (SSH) open? If yes, can I brute force the
password?")
    print("    2. Is port 80 (HTTP) open? If yes, can I intercept plain text
traffic?")
    print("    3. Is there a database port (3306, 5432, 27017) exposed to the
internet?")
    print("    4. Does the server respond to ping? If yes, it's alive and
findable.")
    print("\n✅ That's why BabiEat should run regular Nmap scans on its own
infrastructure.")
if __name__ == "__main__":
    print("=" * 50)
    print("   BabiEat Network Security Audit")
    print("   Student: Norma")
    print(f"   Date: May 2026")
    print("=" * 50)

    # Test with a safe private IP (
    ping_server("10.0.0.5")

    explain_babieat_ports()

    simulate_nmap_scan()

    think_like_an_attacker()

    print("\n" + "=" * 50)
    print("   Audit Complete. Stay secure, BabiEat!")
    print("=" * 50)
```

and got this output:




## What I Learned

Here's the thing that actually stuck with me after doing this task:

**You have to be right 100% of the time. The attacker only has to be right once.**

That's terrifying. I can close every port except one, but if I miss one — just one — someone could get in.

For BabiEat, that means:

- Run Nmap scans every week, not just once

- Don't assume a port is closed just because you never use it
- Check your own servers before someone else does

Also, I learned that cybersecurity is a lot of "what if" thinking. What if someone guesses the SSH password? What if a restaurant owner leaves their laptop unlocked at a cafe? What if a driver connects to fake Wi-Fi?

It's exhausting to think this way all the time. But I guess that's the job.

---

## What BabiEat Should Actually Do

1. **Only keep port 443 (HTTPS) open** — everything else should be closed or behind a VPN
2. **Run weekly Nmap scans** — automate this so a human doesn't have to remember
3. **If port 80 is open, redirect everything to 443** — no plain text traffic allowed
4. **Put SSH (port 22) behind a VPN** — don't expose admin access to the whole internet

---

## Final Thought

Before this task, I thought network security was just about having a strong password. Now I understand it's about visibility. You can't protect what you can't see. Nmap is how we see.

I'm honestly glad I did this hands-on. Reading about port scanning in a textbook is nothing compared to actually running it and seeing open ports appear on your screen. That moment when 10.0.0.5 responded? That's when it clicked for me.

### ***Task 6:*** **Conclusion & Summary of Learning**

Over the course of this internship at BabiEat, I have transitioned from understanding cybersecurity as a theoretical concept to seeing it as a **functional necessity** for a business.

- **Integrated Security:** I learned that encryption (Tasks 2 & 3) is the tool we use to achieve the "Confidentiality" promise of the CIA Triad (Task 1).
- **Trust Mechanics:** Digital Signatures (Task 4) aren't just code; they are the digital equivalent of a physical contract, ensuring that every BabiEat delivery status is authentic.

- **Infrastructure Awareness:** Network security (Task 5) taught me that the "walls" of our digital fortress (Firewalls and Port Management) are just as important as the locks on our data.

**How it Connects:** At BabiEat, these concepts work in harmony. When a customer places an order, **Asymmetric Encryption** secures their payment, **Symmetric Encryption** protects their address in our database, **Digital Signatures** verify their delivery, and **Network Security** ensures the app stays online during the dinner rush. Security is not a barrier to business; it is the engine that builds customer trust.

## ***Task 7:*** Digital Repository & Project Handoff

**1. Project Organization** To ensure this project follows professional software development standards, all documentation, Python scripts, and technical demos have been organized into a structured GitHub repository. This organization allows for version control, easy collaboration, and a centralized "Source of Truth" for BabiEat's security fundamentals.

**2. Repository Link**
https://github.com/nanakaddy666-hub/BabiEat-Cybersecurity-Fundamentals

**3. Repository Structure**

- **`README.md`**: The landing page containing project objectives, setup instructions, and the "How to Run" guide for the security demos.
- **Python Scripts**: Modular files containing the logic for Symmetric/Asymmetric encryption, Digital Signatures, and Network Auditing.
- **Documentation**: The finalized PDF report

**4. The Smart Insight** In a modern tech company, "Code is Documentation." By hosting these tasks on GitHub, we ensure that security practices are transparent and accessible to the engineering team. This moves security out of a static document and into a living, breathing part of the BabiEat ecosystem, reflecting a "Security-as-Code" mindset.